

University of Asia Pacific

Department of Computer Science and Engineering

CSE 402: Operating System Lab

LAB REPORT

Submitted by: Shad Bin Moshiur

Student ID: 23101143

Section: C-2

Lab task: 7

Submitted to:

Atia Rahman Orthi
Lecturer,
Department of Computer Science and Engineering

Date of Submission: 17/10/2026

Lab Task 8: Disk Scheduling — SSTF

(Shortest Seek Time First)

CSE 402: Operating System Lab | Student ID: 23101143

---

1. Objective

To implement the Shortest Seek Time First (SSTF) disk scheduling algorithm in Python and calculate the service order and the total head movement for a given queue of disk track requests.

2. Theory

SSTF is a disk scheduling algorithm that always services the pending request closest to the current head position, i.e. the request with the minimum seek distance. After each service, the head moves to that track and the nearest remaining request is selected again.

- Advantage: much lower total head movement and average seek time than FCFS.
- Disadvantage: requests far from the head can be postponed repeatedly (starvation), and the result is not always globally optimal.

3. Problem Data

The same request set was run twice — once in the given (unordered) sequence and once as a sorted (ordered) queue:

Parameter	Value
Request queue (unordered)	3, 60, 10, 25, 30, 4
Request queue (ordered)	3, 4, 10, 25, 30, 60
Initial head position	41

Table 1: Input to the SSTF program

4. Implementation Summary

- Start with Head = 41, total = 0, and order = [Head].
- While requests remain: pick nearest = min(requests, key = |r − Head|).
- Add |nearest − Head| to total, move the head to nearest, record it in order, and remove it from the queue.
- Finally print the service order and the total head movement.

5. Step-by-Step Trace

Step	From	To (nearest)	Seek Distance
1	41	30	11
2	30	25	5
3	25	10	15
4	10	4	6
5	4	3	1
6	3	60	57
Total	**-**	**-**	**95**

Table 2: Head movement at each step (identical for both queue orders)

6. Results

Queue	Service Order	Total Head Movement
Unordered	41 --> 30 --> 25 --> 10 --> 4 --> 3 --> 60	95
Ordered	41 --> 30 --> 25 --> 10 --> 4 --> 3 --> 60	95

Table 3: SSTF results

Average seek distance per request = 95 / 6 = 15.83 tracks (approx.).

7. Program Output

Console output of the program (code in Section 10):

```
41-->30-->25-->10-->4-->3-->60
Total Head Movement unordered = 95
41-->30-->25-->10-->4-->3-->60
Total Head Movement  Ordered = 95
```

Figure 1: Console output — SSTF service order and total head movement

8. Discussion

Both the unordered and the ordered queue produced the same service order (41 --> 30 --> 25 --> 10 --> 4 --> 3 --> 60) and the same total head movement of 95 tracks. This is because SSTF selects requests by distance from the current head, not by their position in the queue, so the arrival order of the requests does not affect the result here (no ties occurred at any step).

The head first moved downward, serving 30, 25, 10, 4 and 3 in turn, because each was the closest pending request. Request 60 — which was only 19 tracks from the starting head — was left until the very end, and the final jump from 3 to 60 alone cost 57 of the 95 tracks (60% of the total). This shows the main weakness of SSTF: distant requests can be postponed and the head may have to sweep back across the disk, which can lead to starvation if new nearby requests keep arriving.

For reference, serving the same requests in FCFS order (3, 60, 10, 25, 30, 4) from head 41 would need 38 + 57 + 50 + 15 + 5 + 26 = 191 tracks, so SSTF reduces the head movement by about 50% for this data set.

9. Conclusion

The SSTF disk scheduling algorithm was implemented successfully. For the request queue with the head starting at track 41, it gave a total head movement of 95 tracks, independent of the input order. SSTF greatly reduces seek time compared to FCFS, but it can starve far-away requests and is not guaranteed to be optimal, so algorithms such as SCAN or C-SCAN are preferred when fairness matters.

10. Appendix: Python Source Code

The complete Python implementation used for this lab:

```
# -*- coding: utf-8 -*-
"""disk schedule sstf.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1i2hAt_-mGWxRLpD6pF_BxhTbNwk-kIhf
"""

#Unordered
requests = [3,60,10,25,30,4]
Head = 41
total = 0
n = len(requests)
order=[Head]

while requests:
  nearest = min(requests, key = lambda r : abs(r-Head))
  total += abs(nearest - Head)
  Head = nearest
  order.append(Head)
  requests.remove(nearest)


print("-->".join(map(str,order)))
print("Total Head Movement =",total)

requests = [3,4,10,25,30,60]
Head = 41
total = 0
n = len(requests)
order=[Head]

while requests:
  nearest = min(requests, key = lambda r : abs(r-Head))
  total += abs(nearest - Head)
  Head = nearest
  order.append(Head)
  requests.remove(nearest)


print("-->".join(map(str,order)))
print("Total Head Movement =",total)
```